

ResNet Skip Connections & Gradient Flow, Worked Out by Hand

Why $y = x + F(x)$ keeps gradients alive through depth

A complete numerical walk-through of the one trick that made very deep networks trainable. We take a plain deep stack and watch its backward signal collapse to a product of shrinking factors — $0.8^8 = 0.168$, $0.5^{10} = 0.001$ — then add a single identity skip and watch the same signal survive. Every number below was produced by the NumPy/PyTorch reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It derives, by hand, the backward rule of a residual block $y = x + F(x)$, shows that its local Jacobian factor is $1 + F'(x)$, and contrasts that with a plain stack whose Jacobian is a bare *product* of per-layer factors. We then push real numbers through both: a saturated residual block whose $F' = 0$ still passes the upstream gradient *unchanged*, while the plain block at the same saturation passes 0.

How to read this. Each step isolates one mechanism on the cleanest possible numbers, with a *Mini-example* or coloured box where it helps. The single equation to carry away is the boxed $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} (1 + F'(x))$ in Step 2 — everything else is that +1 doing its job over depth.

Concepts covered, in order: the plain deep stack as a chain of Jacobians · why a product of < 1 factors vanishes (0.8^8 , 0.5^{10}) · the residual block $y = x + F(x)$ · its backward rule and the identity +1 · a fully worked single-neuron block, saturated and unsaturated · depth-by-depth survival, plain vs. residual · measured per-layer gradient norms in PyTorch · when to reach for residuals and the shape-matching caveat.

1 The setup: a deep stack and the chain rule

A deep feed-forward network is a composition. Write the input as $x = h_0$ and let each layer transform the previous activation:

$$h_1 = f_1(h_0), \quad h_2 = f_2(h_1), \quad \dots, \quad h_L = f_L(h_{L-1}), \quad y = h_L.$$

Training needs $\partial L / \partial x$ — how the loss responds to the *earliest* activation. The chain rule walks backward through every layer, multiplying one Jacobian factor per layer:

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_L} \cdot \prod_{\ell=1}^L \frac{\partial h_\ell}{\partial h_{\ell-1}}$$

The whole story of vanishing gradients lives in that $\prod$. Each factor $\partial h_\ell / \partial h_{\ell-1}$ is roughly the product of a weight matrix and an activation slope; for stable forward signals those factors tend to sit *below* 1 in magnitude. A product of many numbers below 1 races to zero.

Quantity	Symbol	Meaning
Per-layer Jacobian factor	$\frac{\partial h_\ell}{\partial h_{\ell-1}}$	how layer ℓ scales the backward signal
Plain stack gradient	$\prod_\ell \frac{\partial h_\ell}{\partial h_{\ell-1}}$	a <i>product</i> — collapses if factors < 1
Residual block factor	$1 + F'(x)$	a <i>sum</i> — the 1 is the identity skip

Intuition

Think of the backward pass as a signal handed from the loss back toward the input, re-scaled once per layer. In a plain stack the re-scalings *multiply*: eight quiet 0.8's in a row leave you with 0.168 of the signal; ten 0.5's leave 0.001. A skip connection changes the arithmetic from multiplication toward addition — it adds a clean 1 to each factor, a gradient superhighway that bypasses the lossy block entirely.

2 Step 1 — The plain stack vanishes: a product of small factors

Take the simplest scalar caricature where every layer multiplies the backward signal by the same factor c . Then the product is just a power:

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_L} \cdot c^L.$$

Two cases the course points at, computed exactly:

$$c = 0.8, \quad L = 8 : \quad 0.8^8 = \boxed{0.168}, \quad c = 0.5, \quad L = 10 : \quad 0.5^{10} = \boxed{0.001}.$$

Watch the first one decay, term by term — each layer skims off another fifth of the signal:

$$\begin{aligned} 0.8^1 &= 0.800, & 0.8^2 &= 0.640, & 0.8^3 &= 0.512, & 0.8^4 &= 0.410, \\ 0.8^5 &= 0.328, & 0.8^6 &= 0.262, & 0.8^7 &= 0.210, & 0.8^8 &= 0.168. \end{aligned}$$

By eight layers only 16.8% of the gradient remains; the earliest layer learns at a sixth of the intended rate. The 0.5 case is worse: $0.5^{10} = 0.000977$, i.e. the input gradient is a *thousandth* of the output gradient — effectively frozen.

Watch out

The product cuts both ways. If each factor sits *above* 1 the gradient *explodes*: $1.5^8 = 25.629$, so the input gradient is 25× the output’s, and updates diverge. A bare product is unstable in both directions; it is only ever well-behaved when every factor is pinned suspiciously close to 1 — which is exactly what the identity skip arranges for free.

3 Step 2 — The residual block: factor becomes $1 + F'(x)$

A residual block does not ask a layer to produce the next activation outright. It asks for a *correction* $F(x)$ and adds it to a verbatim copy of the input:

$$y = x + F(x)$$

Differentiate the block’s output with respect to its input. The x term contributes 1; the F term contributes $F'(x)$:

$$\frac{\partial y}{\partial x} = \frac{\partial}{\partial x}(x + F(x)) = 1 + F'(x).$$

Chaining the upstream gradient $\partial L/\partial y$ through the block:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} (1 + F'(x)) = \underbrace{\frac{\partial L}{\partial y}}_{\text{identity path}} + \underbrace{\frac{\partial L}{\partial y} F'(x)}_{\text{through } F}$$

The factor is a *sum*, not a bare product. Even if the block F is badly trained or saturated — $F'(x) \rightarrow 0$ — the factor does not go to 0; it goes to 1, and the upstream gradient $\partial L/\partial y$ passes through *unchanged*. That surviving 1 is the identity skip.

Intuition

A skip connection is a gradient superhighway. The block learns a *residual* correction $F(x)$, not the whole mapping — so the easy default “leave the signal alone” is $F \equiv 0$, which the block can represent exactly. On the forward pass that means a do-nothing block is harmless; on the backward pass it means the +1 guarantees a direct, undamped route for the gradient back to x no matter what F is doing.

Mini-example — this operation in isolation

The per-layer factor for a few residual blocks. With F' the slope of the correction:

$$F' = 0 \Rightarrow 1 + F' = 1.0, \quad F' = +0.2 \Rightarrow 1.2, \quad F' = -0.2 \Rightarrow 0.8, \quad F' = -0.8 \Rightarrow 0.2.$$

Stack eight blocks each with a small $F' = 0.05$: the product is $(1.05)^8 = 1.477$ — mildly amplifying, not vanishing. Stack eight *saturated* blocks, $F' = 0$: the product is $(1.0)^8 = 1.0$ exactly. The identity floor holds the gradient at full strength.

4 Step 3 — One neuron, by hand: the +1 in action

Make it fully concrete with a one-dimensional block $F(x) = w_2 \text{relu}(w_1 x + b_1)$, weights $w_1 = 0.5$, $b_1 = -0.4$, $w_2 = 0.3$, and an upstream gradient $\partial L / \partial y = 2.0$.

Unsaturated input $x = 1.0$. Forward:

$$z = w_1 x + b_1 = 0.5(1.0) - 0.4 = 0.100, \quad a = \text{relu}(z) = 0.100, \quad F(x) = w_2 a = 0.030, \\ y = x + F(x) = 1.0 + 0.030 = 1.030.$$

Backward through the block. Since $z > 0$, $\text{relu}'(z) = 1$, so

$$F'(x) = w_2 \cdot \text{relu}'(z) \cdot w_1 = 0.3 \cdot 1 \cdot 0.5 = 0.150, \quad \frac{\partial y}{\partial x} = 1 + F'(x) = 1.150, \\ \frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} (1 + F'(x)) = 2.0 \times 1.150 = \boxed{2.300}.$$

Saturated input $x = 0.5$. Now $z = 0.5(0.5) - 0.4 = -0.150 < 0$, so the ReLU is dead: $\text{relu}'(z) = 0$ and $F'(x) = 0$. The block contributes *nothing*, yet:

$$\frac{\partial y}{\partial x} = 1 + 0 = 1.000, \quad \frac{\partial L}{\partial x} = 2.0 \times 1.000 = \boxed{2.000} \text{ (survives).}$$

Compare the *plain* block $y = F(x)$ (no skip) at the same saturation: its factor is the bare $F'(x) = 0$, so

$$\left. \frac{\partial L}{\partial x} \right|_{\text{plain}} = \frac{\partial L}{\partial y} F'(x) = 2.0 \times 0 = \boxed{0.000} \text{ (dead).}$$

Same block, same dead ReLU. With the skip the gradient passes at full 2.000; without it the gradient is annihilated. This is the entire mechanism in two numbers.

Watch out

The +1 keeps the *identity path* alive, but it is honest about what it does: alone it propagates $\partial L / \partial y$ *unchanged* — it does not amplify a weak signal, it just refuses to destroy it. The job of F is still to learn useful corrections; the skip only guarantees that a poorly-trained F cannot starve the layers beneath it of gradient.

5 Step 4 — Survival over depth: plain vs. residual

Now scale to depth. Suppose every plain layer scales the backward signal by 0.8, while every residual block has a small correction slope giving factor 0.9 (take $F' = -0.1$), or is fully saturated giving the clean identity factor 1.0. Track the surviving fraction of gradient as depth grows:

depth L	plain 0.8^L	residual 0.9^L ($F'=-0.1$)	residual 1.0^L (identity)
1	0.800	0.900	1.000
2	0.640	0.810	1.000
4	0.410	0.656	1.000
8	0.168	0.430	1.000
16	0.028	0.185	1.000
32	0.001	0.034	1.000

At 32 layers the plain stack retains 0.1% of its gradient — the early layers are effectively untrainable. The residual stack with $F' = -0.1$ retains 3.4%, an order of magnitude better, and the pure-identity column never moves off 1.000. Real residual blocks sit between those last two columns: their F' is small and varied, but every factor carries the +1 floor, so the product cannot collapse the way 0.8^L does.

When this is the right choice

Reach for residual connections whenever you stack *many* layers — in practice past ~ 20 , and certainly for the 50- and 100-layer networks where ResNet was born. Below a handful of layers a plain stack trains fine and the skip buys little. The deeper you go, the more the $\prod$ vs. $1 + F'$ distinction decides whether the early layers ever learn.

6 Step 5 — Measured in PyTorch: per-layer gradient norms

The scalar caricature is borne out by a real autograd run. We build *one* set of 8 linear layers (dimension 16, weights scaled by 0.6, tanh activations) and run the *same* weights two ways: plain ($h \leftarrow \tanh(Wh)$) and residual ($h \leftarrow h + \tanh(Wh)$). We record the norm of the gradient arriving at the input of each layer.

layer (input of)	plain $\ \partial L/\partial h\ $	residual $\ \partial L/\partial h\ $
0 (earliest)	0.000	7.450
1	0.001	6.800
2	0.003	5.708
3	0.009	5.587
4	0.028	5.086
5	0.081	5.111
6	0.237	4.654
7 (latest)	0.931	4.287

Read the plain column top-to-bottom: the gradient is healthy at the last layer (0.931) and decays by more than three orders of magnitude as it reaches the first (0.000466, shown as 0.000). That is the vanishing gradient, measured. The residual column barely droops — 4.287 at the bottom, 7.450 at the top. At the network input the norms are

$$\|\partial L/\partial x\|_{\text{plain}} = 0.000466, \quad \|\partial L/\partial x\|_{\text{residual}} = 7.450, \quad \text{ratio} = \boxed{15976.326}.$$

The residual input gradient is roughly sixteen-thousand times larger — the difference between an early layer that learns and one that is frozen.

Watch out

The skip adds x to $F(x)$, so x and $F(x)$ *must share shape*. When a block changes the channel count or spatial size (a stride-2 down-sampling block), you cannot add the raw x . Project it first with a 1×1 convolution (or a linear layer) on the skip path so the dimensions match: $y = \text{proj}(x) + F(x)$. Forgetting this is the classic “size mismatch” error when wiring a ResNet by hand.

7 Plain vs. residual, at a glance

	Plain stack	Residual stack
Block	$y = F(x)$	$y = x + F(x)$
Local backward factor	$F'(x)$	$1 + F'(x)$
Depth- L gradient	product $\prod F'_\ell$	product $\prod (1 + F'_\ell)$
Saturated block ($F'=0$)	factor 0 — gradient dies	factor 1 — gradient survives
0.8 / 0.9 each, $L=8$	0.168	0.430
Measured $\ \partial L / \partial x\ $	0.000466	7.450
Network learns F as	the whole mapping	a correction (residual)
Shape requirement	—	x and $F(x)$ must match (else project)

8 Equation cheat-sheet

Residual block (forward)	$y = x + F(x)$
Block backward factor	$\frac{\partial y}{\partial x} = 1 + F'(x)$
Gradient through block	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} (1 + F'(x))$
Plain deep stack	$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_L} \prod_\ell \frac{\partial h_\ell}{\partial h_{\ell-1}}$
Constant-factor vanish	c^L ($0.8^8=0.168$, $0.5^{10}=0.001$)
Constant-factor explode	$1.5^8 = 25.629$
Saturated residual	$F' = 0 \Rightarrow \text{factor} = 1$, gradient unchanged
Shape-change skip	$y = \text{proj}_{1 \times 1}(x) + F(x)$

Appendix A Reference implementation

Every number above is reproducible from the following program. The first block is the single-neuron derivation of Step 3; the second measures per-layer gradient norms, plain vs. residual, with shared weights.

```
import numpy as np, torch

# --- Step 1: plain stack as a product of constant factors ---
print(0.8**8)      # 0.16777216 -> 0.168
print(0.5**10)     # 0.0009765625 -> 0.001
print(1.5**8)      # 25.62890625 -> 25.629 (explodes)

# --- Step 3: one-neuron residual block, the +1 in action ---
w1, b1, w2 = 0.5, -0.4, 0.3
dLdy = 2.0
def block(x):
    z = w1*x + b1
    a = max(z, 0.0)          # relu
    F = w2*a
    relup = 1.0 if z > 0 else 0.0
    Fprime = w2*relup*w1     # dF/dx
    y = x + F
    dLdx = dLdy*(1.0 + Fprime) # residual: factor (1 + F')
    dLdx_plain = dLdy*Fprime   # plain (no skip): factor F'
    return y, Fprime, dLdx, dLdx_plain

print(block(1.0))  # unsaturated: F'=0.15, dL/dx=2.3   (plain 0.30)
print(block(0.5))  # saturated:   F'=0.00, dL/dx=2.0   (plain 0.00, DEAD)

# --- Step 5: per-layer grad norms, plain vs residual (same weights) ---
torch.manual_seed(0)
dim, nL, scale = 16, 8, 0.6
layers = []
for _ in range(nL):
    lin = torch.nn.Linear(dim, dim, bias=False)
    with torch.no_grad():
        lin.weight.mul_(scale)
    layers.append(lin)

def run(residual, seed=1):
    torch.manual_seed(seed)
    x = torch.randn(1, dim, requires_grad=True)
    h, inputs = x, []
    for lin in layers:
        inputs.append(h)
        out = torch.tanh(lin(h))
        h = h + out if residual else out  # <- skip connection
        h.retain_grad()
    h.sum().backward()
    norms = [float(t.grad.norm()) for t in inputs]
    return float(x.grad.norm()), norms

xp, np_norms = run(False)
xr, nr_norms = run(True)
print("plain per-layer:", [round(v, 3) for v in np_norms])
print("resid per-layer:", [round(v, 3) for v in nr_norms])
print("input grad plain =", xp, " residual =", xr)
```

```
print("ratio residual/plain =", xr/xp)  # ~15976.326
```

— end of document —